

CMSE 801 - Day 13

Agent-based Modeling

October 17, 2022

General Course Announcements

- The midterm exam is Wednesday!
- Homework #3 has been released and is due Oct 26 at 11:59 p.m.

Questions/comments from Day 13 PCA

- Can we go over the code that was provided in this assignment?
- I think when it comes to writing code for moving individuals to improve happiness, I need to understand the "order of moves" better. In what order are we supposed to assess individuals?
- Could you go over this pre-class in the class on Monday?
- Many of the problems proposed result in a combination of comparing a value (in a array/list) with the previous or the next, sometimes intervals like this assignment. Is there an efficient piece of "standard" code for such common tasks or everyone comes up with their own as required?
- Can you give more explanation for the rule number 3, ie "Anyone who becomes discontent gets a turn after the originally unhappy people get to move".
- Explain schelling's model and how to count neighbors from the end
- I'm still confused on how to know when a round is finished?
- I feel that if the tolerance is too high, the people will never get satisfied (civil war starts!). But when I run it in 2D model, although the moving never stops but the dynamic segregation rate keeps at a very low value.
- Does the difference between list and numpy array really matter in describing the game board?
- Schelling's model was very unclear to me. Could you go over agent-based modeling and how to interpret changes in these models again?
- You mentioned needing to be familiar with JupyterHub for the mid-term. Does that refer to the Jupyter notebook system we use for all assignments, or is it some other Jupyter component?
- It's going okay in overall, but so scared for the Mid-term examination.

```
In [1]: import numpy as np
```

```
In [2]: def initialize_board(N_people=32,seed=1234):
        np.random.seed(seed)
        return np.random.choice([0,1],N_people)

        # To make sure the output changes, we use a random number for the seed
        board = initialize_board(24,seed=np.random.randint(0,1024))
        print(board)
```

```
[1 0 0 0 0 0 0 0 1 0 1 0 0 1 0 1 0 0 0 0 0 0 0 1]
```

```
In [3]: def is_happy(my_list, my_index, tol_prob = 0.5, neighbor_N = 4):
        '''
        This function take a game board (my_list) and checks to see if the agent
        at the given index (my_index) is "happy" or "unhappy" based on a provided
        tolerance (tol_prob) and a given neighborhood size (neighbor_N).
        It returns 'True' if happy and 'False' if unhappy under those circumstances.
        '''

        # do some error-checking (is the index within the allowed range?)
        if my_index < 0 or my_index > len(my_list):
            print("you've made an indexing error!", my_index)
            return

        # store the value we will compare neighbors to (e.g. 0 or 1)
        my_value = my_list[my_index]

        start = max(my_index - neighbor_N, 0) # start 4 to the left
        end = min(my_index + neighbor_N, len(my_list)-1) # end 4 to the right

        # counter for keeping track of the neighbors that are like me
        neighbors_like_me = 0

        # counter for keeping track of total neighbors
        total_neighbors = 0

        # loop over the specified range
        for i in range(start,end+1):
            if i == my_index:
                continue # don't count myself
            if my_list[i] == my_value: # if this neighbor is like me, keep track of
                neighbors_like_me += 1
            total_neighbors+=1 # also keep track of total neighbors

        # happy if at least half are like me, unhappy otherwise
        # note: it's at least half because we're not double-counting our
        # own value
        if neighbors_like_me/total_neighbors >= tol_prob:
            return True
        else:
            return False
```

```
In [4]: world = initialize_board()
        print(world)
        print(is_happy(world,5))
```

```
[1 1 0 1 0 0 0 1 1 1 1 0 0 1 0 0 0 0 0 0 0 0 1 0 1 1 0 0 1 0]
False
```

```
In [5]: print(is_happy(world,7))
```

```
True
```

